CSS exercises

Instructions

Use VS Code and a browser for these exercises.

For each exercise that contains two files:

- Create the files using the exact names shown.
- Keep `index.html` and `styles.css` in the same folder.
- Open `index.html` through Live Server.
- Do not add JavaScript.
- Do not use Tailwind CSS.

Complete each exercise independently unless it refers to an earlier part of the same exercise.

Section A: CSS setup, selectors, and basic styling

Exercise 1: Repair the stylesheet connection

Create these files:

```
exercise-01/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Reading room</title>

  <link
    rel="stylesheet"
    href="./style.css"
  >
</head>
<body>
```

```
  <main class="room-card">
    <h1>Reading room</h1>

    <p>
      The room is open from 9 AM to 6 PM.
    </p>
  </main>
</body>
</html>
```

`styles.css`

```
body {
  background-color: #f5f5f4;
  color: #292524;
}

.room-card {
  padding: 24px;
  background-color: white;
  border: 1px solid #a8a29e;
}
```

The browser shows the HTML, but none of the CSS appears.

Find and correct the problem.

Then explain:

1. Which file contains the incorrect reference?
2. Why does the page still display its HTML?
3. Why does the CSS fail to load?

Exercise 2: Choose selectors that match the requirement

Create these files:

```
exercise-02/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Workshop schedule</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <header id="page-header">
    <h1>Weekend workshops</h1>

    <p class="intro">
      Select a session to view its details.
    </p>
  </header>

  <main>
    <section class="workshop-list">
      <article class="workshop-card">
        <h2>HTML foundations</h2>
        <p>Saturday, 10 AM</p>
      </article>

      <article class="workshop-card">
        <h2>Responsive CSS</h2>
        <p>Sunday, 10 AM</p>
      </article>
    </section>
  </main>
</body>
</html>
```

Write selectors that apply these requirements:

1. Remove the default margin from every element using the universal selector.
2. Give the element with the ID `page-header` a light blue background.
3. Make only the paragraph with the class `intro` dark blue.
4. Give every element with the class `workshop-card` a border.
5. Make every `<h2>` inside `workshop-list` larger.
6. Change only paragraphs that are direct children of `workshop-card`.

Use a different selector for each requirement.

Exercise 3: Predict which colour is applied

Read this HTML:

```
<p
  id="registration-message"
  class="notice"
>
  Registration closes today.
</p>
```

Read this CSS:

```
p {
  color: green;
}

.notice {
  color: darkorange;
}

#registration-message {
  color: darkred;
}
```

Answer these questions before testing the code:

1. What colour will the message use?
2. Which selector has the highest specificity?
3. Would moving the `p` rule below the ID rule change the final colour?
4. What would happen if the ID rule were removed?

Then test your answers in the browser.

Exercise 4: Style link interaction states

Create these files:

```
exercise-04/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Course navigation</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <nav class="course-nav">
    <a href="#overview">Overview</a>
    <a href="#schedule">Schedule</a>
    <a href="#contact">Contact</a>
  </nav>
</body>
</html>
```

Write CSS for these link states:

- An unvisited link should be dark blue.
- A visited link should be purple.
- A link under the pointer should have an underline.
- A link being clicked should be red.

Use:

```
:link
:visited
:hover
:active
```

Apply the pseudo-classes only to links inside `course-nav`.

Then explain which state applies:

1. Before the link has been visited
2. After the link has been visited
3. While the pointer is over the link

4. While the mouse button is pressed

Section B: Box model, sizing, display, and overflow

Exercise 5: Add space in the correct place

Start with this HTML:

```
<section class="notice">
  <h2>Room change</h2>

  <p>
    The workshop will now be held in Lab 3.
  </p>
</section>

<section class="notice">
  <h2>Bring a laptop</h2>

  <p>
    A browser and code editor are required.
  </p>
</section>
```

The two notices already have a border:

```
.notice {
  border: 2px solid #78716c;
}
```

Apply these changes:

- Add `20px` of space between the content and the border.
- Add `16px` of space below each notice.
- Keep the two types of spacing separate.

Then answer:

1. Which property adds space inside the border?
2. Which property separates one notice from the next?
3. What would happen if both values were written as padding?

Exercise 6: Repair an incorrect display rule

Create these files:

```
exercise-06/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Resource links</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <nav class="resource-nav">
    <a href="#">Documentation</a>
    <a href="#">Exercises</a>
    <a href="#">Support</a>
  </nav>
</body>
</html>
```

`styles.css`

```
.resource-nav a {
  position: block;
  margin-bottom: 12px;
}
```

The links do not begin on separate lines.

Correct the rule so that each link behaves as a block-level element.

Then answer:

1. Which CSS property was incorrect?
2. Which value should be used?
3. Why is `block` a value rather than a property name?
4. What visible change should appear after the correction?

Exercise 7: Control overflowing content

Create these files:

```
exercise-07/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Announcement</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <section class="announcement">
    <h1>Important announcement</h1>

    <p>
      The centre will remain open for extended
      hours during the examination period.
      Learners may use the reading room, computer
      room, and discussion room until 9 PM.
      Please carry your registration card and
      follow the room capacity limits.
    </p>
  </section>
```

```
</body>
</html>
```

`styles.css`

```
.announcement {
  width: 280px;
  height: 100px;
  padding: 16px;
  border: 2px solid #57534e;
}
```

The text extends beyond the available height.

Change the CSS so that:

- The element keeps its current height.
- The complete text remains available.
- A scrollbar appears only when the content needs it.

Then explain why `overflow: hidden` would not meet the requirement.

Section C: Positioning and stacking

Exercise 8: Predict static positioning

Read this HTML and CSS:

```
<div class="promotion">
  Registration is open.
</div>
```

```
.promotion {
  position: static;
  right: 40px;
  bottom: 20px;
  padding: 12px;
  border: 2px solid orange;
}
```

Answer these questions before running the code:

1. Where will the promotional message appear?
2. Will `right: 40px` move it to the left?
3. Will `bottom: 20px` move it upward?

4. Why do the offset properties have no effect?

Then change only the `position` value so that the offsets can move the element relative to its original position.

Exercise 9: Move an element without removing its original space

Create these files:

```
exercise-09/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Course badge</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <article class="course-card">
    <span class="course-badge">
      New
    </span>

    <h1>Responsive web design</h1>

    <p>
      Learn how layouts respond to different
      viewport sizes.
    </p>
  </article>
</body>
</html>
```

Write CSS that moves `course-badge`:

- `8px` to the right
- `6px` upward
- Relative to its original position

The badge should continue to reserve its original space in the document.

Then explain why `position: relative` fits this requirement better than `position: absolute`.

Exercise 10: Position a label inside its card

Create these files:

```
exercise-10/
├── index.html
└── styles.css
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Live session</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <article class="session-card">
    <span class="live-label">
      Live
    </span>

    <h1>CSS positioning clinic</h1>

    <p>
      The session begins at 5 PM.
```

```
    </p>
  </article>
</body>
</html>
```

`styles.css`

```
.session-card {
  width: 320px;
  padding: 24px;
  border: 2px solid #57534e;
}

.live-label {
  position: absolute;
  top: 8px;
  right: 8px;
}
```

The label is not positioned relative to the card.

Correct the CSS so that the label appears in the top-right corner of `session-card`.

Then explain:

1. Which element is absolutely positioned?
2. Which element becomes its positioning reference?
3. Why is the missing declaration added to the parent?

Exercise 11: Choose between fixed and sticky

Write the essential CSS for both requirements.

Support button

A support button should:

- Stay in the bottom-right corner of the viewport
- Remain visible while the page scrolls
- Sit `20px` from the right and bottom edges

Use this class:

```
<button class="support-button">
  Help
</button>
```

Section heading

A heading inside a long page section should:

- Scroll normally until it reaches the top of the viewport
- Remain near the top while the rest of the section continues to scroll

Use this class:

```
<h2 class="section-heading">
  Course schedule
</h2>
```

After writing the CSS, explain why one requirement uses `fixed` and the other uses `sticky`.

Exercise 12: Repair the stacking order

Read this HTML and CSS:

```
<nav class="navigation">
  <button>Courses</button>

  <div class="dropdown">
    <a href="#">Frontend</a>
    <a href="#">Backend</a>
  </div>
</nav>

<section class="banner">
  New courses available
</section>
```

```
.navigation {
  position: relative;
  height: 40px;
}

.dropdown {
  position: absolute;
  top: 30px;
  left: 0;
  width: 160px;
  padding: 12px;
  background-color: white;
  border: 1px solid gray;
  z-index: 1;
}
```

```css
.banner {
  position: relative;
  padding: 24px;
  background-color: lightblue;
  z-index: 2;
}
```

The banner appears above the dropdown where they overlap.

Complete these tasks:

1. Identify why the banner appears on top.
2. Change one `z-index` value so that the dropdown appears above the banner.
3. Explain why adding more padding to the dropdown would not solve the stacking problem.
4. Explain why `display: block` would not solve it either.

Section D: Responsive units and media queries

Exercise 13: Choose a responsive unit

Choose from:

```
%
rem
vw
vh
em
```

Use each requirement to select a suitable unit.

1. A content area should use `90` of its parent element's width.
2. A page heading should be sized relative to the root font size.
3. A hero section should occupy half of the viewport height.
4. Horizontal spacing should equal the full viewport width divided by twenty.
5. Button padding should change when the button's own font size changes.

Write one CSS declaration for each requirement.

Then state what each selected unit is relative to.

Exercise 14: Repair a media query

The following media query does not work:

```
media max-width = 600px {
  .notice {
    font-size = 1rem;
  }
}
```

Correct the syntax so that `.notice` uses `1rem` when the viewport is `600px` wide or narrower.

Identify each correction you made.

Exercise 15: Predict breakpoint behaviour

Read this CSS:

```
.notice {
  font-size: 1.25rem;
  padding: 1.5rem;
}

@media (max-width: 600px) {
  .notice {
    font-size: 1rem;
  }
}
```

State the font size and padding at each viewport width:

1. `540px`
2. `600px`
3. `720px`

Then answer:

1. Does the media query change the padding?
2. Is the condition true at exactly `600px`?
3. Which rule supplies the padding at every width?

Exercise 16: Rewrite the layout using a mobile-first approach

The following stylesheet begins with wider-screen styles:

```
.course-nav a {
  display: inline;
  margin-right: 16px;
}
```

```css
.course-title {
  font-size: 3rem;
}

@media (max-width: 767px) {
  .course-nav a {
    display: block;
    margin-right: 0;
    margin-bottom: 12px;
  }

  .course-title {
    font-size: 2rem;
  }
}
```

Rewrite it using a mobile-first approach.

Apply these requirements:

- Navigation links should be block elements on narrow screens.
- Navigation links should appear inline at `768px` and wider.
- The course title should use `2rem` on narrow screens.
- The course title should increase to `3rem` at `768px` and wider.
- Use a `min-width` media query.

Then explain which rules form the base layout.

Final exercise: Build a responsive course information page

Create these files:

```
css-final/
├── index.html
└── styles.css
```

Use the prepared HTML. Do not remove or rename any class or ID.

`index.html`

```html
<!DOCTYPE html>
<html lang="en">
<head>
```

```
  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>Frontend course</title>

  <link
    rel="stylesheet"
    href="./styles.css"
  >
</head>
<body>
  <header class="site-header">
    <a
      class="site-name"
      href="#"
    >
      Community Learning
    </a>

    <nav class="course-nav">
      <a href="#overview">Overview</a>
      <a href="#schedule">Schedule</a>
      <a href="#support">Support</a>
    </nav>
  </header>

  <main class="page-content">
    <section
      id="overview"
      class="course-hero"
    >
      <p class="course-label">
        Beginner course
      </p>

      <h1 class="course-title">
        Frontend foundations
      </h1>

      <p class="course-summary">
        Learn HTML structure, CSS styling,
        responsive design, and Tailwind CSS.
      </p>

      <a
        class="registration-link"
        href="#schedule"
```

```
      >
        View schedule
      </a>
    </section>

    <aside class="course-notice">
      Bring a laptop with a browser and
      code editor installed.
    </aside>

    <section
      id="schedule"
      class="schedule-section"
    >
      <h2>Course schedule</h2>

      <article class="session-card">
        <span class="session-status">
          Open
        </span>

        <h3>HTML and CSS</h3>

        <p>
          Saturday, 10 AM
        </p>
      </article>

      <article class="session-card">
        <span class="session-status">
          Open
        </span>

        <h3>Responsive design</h3>

        <p>
          Sunday, 10 AM
        </p>
      </article>
    </section>
  </main>

  <button class="support-button">
    Help
  </button>

  <footer
    id="support"
    class="site-footer"
  >
    Contact the course team for support.
```

```
  </footer>
</body>
</html>
```

Part 1: Set the page foundation

Write CSS that:

- Removes the default body margin.
- Uses Arial or another sans-serif font.
- Gives the page a light background and dark text.
- Keeps the header, main content, and footer at `90%` of their parent width.
- Prevents those areas from becoming wider than `900px`.
- Centres them horizontally.

Use a percentage for the flexible width.

Part 2: Style the header and navigation

Apply these requirements:

- Add vertical padding to the header.
- Make the site name bold.
- Remove the underline from the site name.
- Display navigation links as blocks on narrow screens.
- Add spacing below each navigation link.
- Change the link colour when the pointer is over it.

Part 3: Style the hero section

Apply these requirements:

- Use a pale blue background.
- Add padding using `rem`.
- Give the hero a minimum height of `50vh`.
- Style the small course label differently from the main title.
- Use `rem` for the title size.
- Add spacing above the registration link.
- Make the registration link behave like a block-level element on narrow screens.

Part 4: Style the notice and session cards

Notice

- Add padding.
- Add a visible left border.
- Add a pale background.
- Add margin above and below the notice.

Session cards

- Add padding, a border, and margin below each card.
- Position each status label in the top-right corner of its card.
- Make the card the positioning reference for the label.
- Add enough top padding so the label does not cover the heading.

Part 5: Add the support button

The support button should:

- Remain in the bottom-right corner of the viewport
- Stay visible while the page scrolls
- Sit `20px` from the right and bottom edges
- Appear above ordinary page content

Use fixed positioning and `z-index`.

Part 6: Add a mobile-first media query

At `768px` and wider:

- Display the navigation links inline.
- Remove their bottom margin.
- Add space to the right of each link.
- Increase the hero padding.
- Increase the course title size.
- Allow the registration link to behave as an inline element.

Use a `min-width` media query.

Completion checks

Test the page at a narrow width and at a width above `768px`.

Confirm that:

- The external stylesheet loads.
- The content width adjusts with the viewport.
- The content stops growing after `900px`.
- Navigation links stack on narrow screens.
- Navigation links appear on one line at wider sizes.
- The hero uses responsive units.
- The status labels stay inside their cards.
- The support button remains attached to the viewport.
- No content is hidden or cut off.
- No Tailwind classes or JavaScript have been added.